Graph DSA

Problem 1:- Social Network Connections

1. Graph Representation

Use an Adjacency List.

Why?

Save memory

Easy to find friends

Fast operations

Representation of given data -

Alice → Bob, Charlie

Bob → Alice, David

Charlie → Alice, Eve

David → Bob, Eve.

Eve → Charlie, David.

2. Find all friends of Alice

Algorithm

1. Go to Alice in adjacency list.
2. Print all connected users.

Result

Friends of Alice = Bob, Charlie.

3. Check if Bob and Eve are directly connected

Algorithm

1. Go to Bob's list.
2. If Eve is present → Yes
3. Else → No.

Result

Bob's friends = Alice, David.

Eve is NOT present ∴ Not directly connected.

4. Shortest Path (Degree of Separation). (BFS)

Algorithm

1. Start from Alice.
2. Visit all neighbours.
3. Then visit neighbours of neighbours.
4. Stop when Eve is found.

Step by Step

Start: Alice → Bob, charlie.

From Bob → David

From Charlie → Eve

Shortest Path

Alice → Charlie → Eve.

Degree of Separation

= 2

Problem 2.

1. Graph Representation

Use a directed Graph (Adjacency List)

Representation

CS101 → CS102, CS201

CS102 → CS 202.

CS 201 →

CS202 →

MATH 101 → CS201

2. Cycle Detection

Use DFS.

Algorithm

1. Mark all nodes as unvisited.
2. For each node:
   a) Visit node

b) Mark it in recursion stack.

c) visit neighbours.

3. Remove node from stack after visiting.

Result

In given graph → No cycle (valid structure).

3. Courses required before CS202

Idea

Find all courses that lead to CS202

Steps

CS202 depends on → CS102

CS102 depends on → CS101

Final Answer

CS101 → CS102 → CS202

Required Courses

CS101, CS102.

4. Topological Sort

Idea

Order courses so prerequisite come first.

Simple Method.

① CS101 – 0
Math101 – 0
CS102 – 1
CS201 – 2
CS202 – 1

② CS101, MATH101

③ CS101 → MATH101 → CS102 → CS201 → CS202

Problem 3 :-

① Graph representation

Directed & weighted graph.

Adjacency list

A → B(5), D(7)

B → C(3)

C → B(3), E(4)

D → A(7), E(2)

E → None.

② Find all intersection Reachable from A.

Using DFS (simple)

```
fxn DFS(node):
    mark node as visited
        print(node)

    for each neighbour:
        if not visited:
            DFS(neighbour).
```

Traversal

A → B → C → E → D.

③ BFS

finds path with minimum edges (turns), not distance.

Start: A

visit neighbours B, D

from B: C

from D: E.

Path → A → D → E

④ DFS might not give shortest path

It goes deep first, not optimal.

It does not care about shortest path.

Problem 4: Island Counters.

1. Graph Modeling

grid - graph.

Vertices : each cell with value 1.

Edges : connect two vertices if they are adjacent up, down, left, right.

So each island is connected component in a graph.

②. DFS Algorithm

```
fun countIsland (grid).
    count = 0
    for i in range (rows):
        for j in rang (cols):
            if grid [i][j] = 1
                DFS[i, j]
                count += 1;
    return count.
fun DFS (i, j)
    if (i < 0 or j < 0 or i >= rows or j >= cols)
        return.
    if grid[i][j] == 0;
        return.
    grid [i] [j] = 0.
    DFS ( i+1, j)
    DFS (i-1, j)
    DFS (i, j+1)
    DFS (i, j-1).
```

③ BFS Algorithm

```
for BFS(i, j)
    queue = [(i, j)]
    grid[i][j] = 0
    while queue not empty:
        (x, y) = queue.pop(0)
        - for each direction (up, down, left, right):
            nx, ny = neighbour
            if valid and grid[nx][ny] == 1:
                queue.append((nx, ny))
                grid[nx][ny] = 0.
```

④ Time & Space complexity

Time = O (rows x cols)

Space

DFS = O(row x cols) recursive

BFS = O(row x cols) queue.

Problem 5 - Network Packet Routing

① Graph representation:

Undirected, unweighted graph.

Adjacency list

$R_1 \to R_2, R_3$

$R_2 \to R_1, R_4$

$R_3 \to R_1, R_4$

$R_4 \to R_2, R_3, R_5$

$R_5 \to R_4, R_6$

$R_6 \to R_5$

② Space complexity

V = no. of vertices
E → no. of edges.

Space ≈ $O(V^2)$ - Adjacency matrix

Space = $O(V+E)$ - Adjacency list.

③ Check if Network is connected

Algorithm (DFS)

```
fun Is connected(graph):
    visited = set()
    DFS(R1)
    If size(visited) == total nodes:
        return True.
    else:
        return false.
```

④ If R4 - R5 fails)

Remove edge
R4 - R5

Now graph.

R1 - R2 - R4 R5 - R6
\ R3 /

No path b/w R1 to R6

Algorithm to check

Remove edge (R4, R5)
Run BFS from R1
If R6 not reached → No path exists.

⑤ Minimum Hops (R1 → R6)

Start = R1
→ R2, R3
→ R4
→ R5
→ R6

Path ⇒ R1 → R2 → R4 → R5 → R6

4 hops